

Implementación de un Sistema de Gestión de Base de Datos en C++

Base de Datos 2 – Proyecto 1

Joaquin Adrian Lopez del Carpio Diego Alonso Figueroa Winkelried José Daniel Grayson Tejada

Joaquín Alberto Mercado Guerra Mathias Rolando Castro Alva

10. May 2026

Índice

<u>1</u>	<u>Introducción</u>	<u>3</u>
<u>2</u>	<u>Objetivo del proyecto</u>	<u>3</u>
<u>3</u>	<u>Arquitectura del proyecto</u>	<u>3</u>
3.1	Componentes principales	4
3.1.1	File manager	4
3.1.2	Buffer manager	5
3.1.3	Layouts	5
3.1.4	Files	6
3.1.5	Table	6
3.1.6	Catalog	6
3.1.7	Query executor	6
3.1.8	API Server	6
3.2	Técnicas adicionales de implementación	7
3.2.1	Cursores	7
3.2.2	API de serialización/deserialización a bytes	7
3.2.3	Uso de features de C++ moderno	8
<u>4</u>	<u>Técnicas implementadas</u>	<u>9</u>
4.1	Parsing	9
4.1.1	Escaneo	10
4.1.2	Parseo	11
4.2	Archivos de registros	11
4.2.1	SeqFile	11
4.2.2	AppendFile	12
4.3	Índices	12
4.3.1	Extendible Hashing	12
4.3.2	B+ Tree	13
4.3.3	R-Tree	13
<u>5</u>	<u>Resultados y análisis</u>	<u>13</u>
5.1	I/O y tiempos de inserción	14
5.2	I/O y tiempos de búsqueda	14
5.2.1	Análisis	16
5.3	Conflictos por concurrencia	16
<u>6</u>	<u>Evidencias de la interfaz</u>	<u>17</u>
<u>7</u>	<u>Anexo</u>	<u>20</u>
7.1	Consulta de ejemplo	20
7.2	Consultas para crear tablas con el dataset de Kaggle	20
7.3	Consultas ejecutadas en la demostración de acceso concurrente	21
7.3.1	Primera Consulta	21
7.3.2	Segunda Consulta	21
7.4	Consultas ejecutadas en la demostración de la interfaz	22

1 Introducción

En 1970, Edgar F. Codd, motivado por la observación de que las computadoras deberían buscar información no por enlaces, sino por contenido, propuso el modelo relacional. Tomó hasta mediados de los años 80 para que las computadoras de uso general fuesen lo suficientemente capaces para desplegar de forma más extensa sistemas de gestión de base de datos (DBMS) basados en la propuesta de Codd. Hacia inicios de los años 1990 y hasta la actualidad, los DBMS relacionales han permanecido el estándar dominante en la industria de las bases de datos. Los DBMS relacionales son, por ende, un pilar fundamental de los sistemas computacionales al día de hoy.

Los DBMS relacionales se fundamentan en un gran número de algoritmos y estructuras de datos diseñados para este dominio específico. Estas técnicas incluyen, por ejemplo, estructuras para guardar registros en archivos (e.g. *sequential file*) y técnicas de indexación de registros. Desde un punto de vista pedagógico, el entendimiento de estas técnicas no solo ofrece un contexto diferente al cual aplicar teoría de algoritmos y estructuras de datos, sino que permite un uso más eficiente e informado de DBMS existentes. El desarrollo y optimización de los DBMS es al día de hoy un campo de estudio activo.

El presente informe desarrolla el diseño de *Koakuma*, una implementación en C++ de un DBMS que se aproxima a los DBMS relacionales y aplica técnicas de gestión de base de datos vistas en clase.

2 Objetivo del proyecto

El presente proyecto tiene el objetivo general de implementar un DBMS funcional que haga uso de las técnicas aprendidas en clase.

En particular, el proyecto tiene los siguientes objetivos específicos:

- Implementar un parser de SQL con soporte para sentencias básicas como `SELECT * FROM ...`, `INSERT INTO ...` y `DELETE FROM ...`.
- Implementar gestión de archivos eficiente mediante la técnica de *sequential file*.
- Implementar índices B+ Tree, extendible hashing y R-Tree.
- Implementar un ejecutor de consultas que haga uso de las estructuras de archivo e índices implementados.
- Desarrollar una interfaz gráfica mediante la cual escribir y realizar consultas.
- Permitir la visualización del índice R-Tree y sus operaciones.
- Evaluar la existencia de conflictos generados por consultas concurrentes en el sistema.

El DBMS implementado admitirá datos de tipo `int`, `real`, `bool`, `varchar` (texto de tamaño arbitrario) y `point2d` (un punto en 2 dimensiones). Debe permitir al usuario crear tablas con las columnas y tipos de atributo que quiera. En particular, las pruebas del sistema serán realizadas con un dataset (detallado más adelante) recuperado de Kaggle sobre viajes de Taxi en la ciudad de Nueva York.

3 Arquitectura del proyecto

El proyecto se puede encontrar con los siguientes enlaces:

- **DBMS/backend:** <https://github.com/project-sdm/koakuma>
- **Frontend web:** <https://github.com/project-sdm/koakuma-frontend>

El proyecto hace uso de las siguientes librerías de C++:

- [yhirose/cpp-http-lib](#): Servidor HTTP del proyecto.
- [nlohmann/json](#): Serialización de JSON para el servidor HTTP.
- [d99kris/rapidcsv](#): Parseo de archivos CSV para creaciones de tabla con `FROM FILE`.
- [Neargye/magic_enum](#): Utilidades de reflexión para enums de C++ (iteración sobre variantes en tiempo de ejecución, obtención de nombres de variantes como strings, etc.).

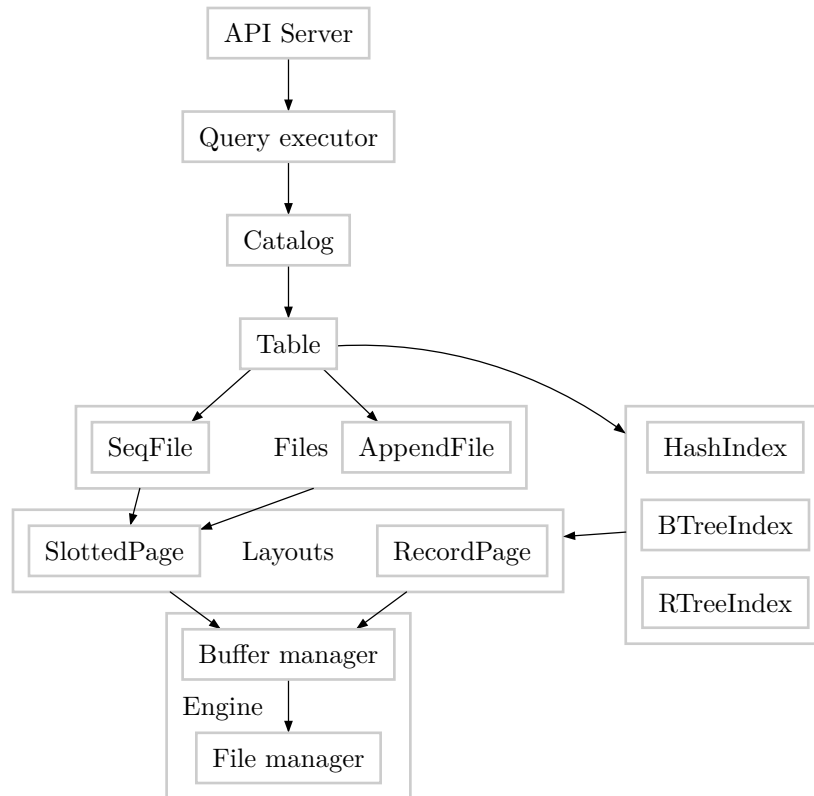


Figura 1: Niveles de abstracción en Koakuma.

3.1 Componentes principales

La Figura 1 muestra una vista de alto nivel de los componentes principales del proyecto. La ausencia de una flecha entre dos componentes no necesariamente significa que no interactúen directamente; este diagrama muestra las capas de abstracción del proyecto.

3.1.1 File manager

El file manager (**FileManager**) gestiona el ciclo de vida de los archivos: abrirlos, cerrarlos, escribir y leer. Permite escrituras y lecturas únicamente por página. Además, se encarga de alojar y desalojar páginas mediante una implementación de free para las páginas. Impone un header común en la primera página de los archivos con la información del free list, y admite la presencia de un header adicional («user header» en el código) para almacenar información de tipos específicos de archivos.

Como el file manager se encarga de gestionar la free list de páginas de cada archivo y vive por debajo del buffer manager (por lo que no puede hacer uso de este último), tiene su propio buffer con capacidad para una página. Toda escritura y lectura referente a la cabecera de un archivo y la free list no pasa por el buffer manager, sino que se hace directamente por el file manager.

Esta decisión de diseño fue tomada a inicios del proyecto. Al tiempo presente, parece ser una mejor idea que el alojamiento de páginas no sea responsabilidad del file manager, sino de algún nuevo componente por encima del buffer manager (para poder hacer uso de este último). Esta refactorización estructural queda como pendiente a futuro.

3.1.2 Buffer manager

El buffer manager (`BufferManager`) gestiona el uso de RAM para almacenar, modificar páginas y escribirlas de vuelta. Usa una política de **LRU** para liberar frames a demanda.

Una abstracción particularmente importante en el proyecto es la clase `PageGuard`: un acceso hacia un frame en memoria. El `PageGuard`:

- Contiene métodos para acceder a la data de la página en RAM.
- Gestiona automáticamente el incremento y decremento del `pin_count` del frame mediante RAII usando su constructor y destructor.
- Modifica la flag `is_dirty` automáticamente cuando se accede a la data de forma no-constante.

Como ejemplo, un `PageGuard` se puede usar de la siguiente manera:

```
{
    PageGuard page = buf_mgr.fetch_page(fid, 2); // `pin_count++`

    // Establecer los 2 primeros bytes de la página a 0x12, 0x34
    page.data()[0] = 0x12; // Usar `data()` activa la flag `is_dirty`
    page.data()[1] = 0x34;

} // `pin_count--` y escribir a disco si `pin_count == 0`
```

Esta abstracción ha sido crucial a lo largo del proyecto para asegurar la liberación automática de páginas y una correcta gestión del atributo `pin_count` de los frames.

El buffer manager del proyecto está configurado con una capacidad de 256 frames para páginas de 4 KB cada una. Esto significa que el buffer manager utiliza exactamente 1 MB de memoria RAM. Se podría aumentar aún más esta capacidad, pero 256 frames ha demostrado ser un valor lo suficientemente cómodo para tener el rendimiento esperado.

3.1.3 Layouts

Los layouts son una abstracción por encima del `PageGuard`: fuerzan una interpretación específica de los bytes crudos de una página. En otras palabras, imponen un significado particular a la data de una página. El proyecto implementa dos layouts:

1. **SlottedPage**: Permite almacenar una lista de «slots» con data asociada de tamaño variable. Los índices B+ Tree y Hash hacen uso de este layout para sus nodos y buckets respectivamente, lo cual permite que estos índices admitan llaves de tamaño variable. El **SlottedPage** compacta sus contenidos automáticamente **a demanda**: un usuario del **SlottedPage** solo puede saber su espacio libre total (contando huecos en la sección de data), y el **SlottedPage** se encarga de compactar automáticamente antes de una inserción cuando sea 100% necesario.
2. **RecordPage**: Similar a **SlottedPage**, pero almacena data de tamaño fijo. Está parametrizado para tomar el tipo de dato a almacenar.

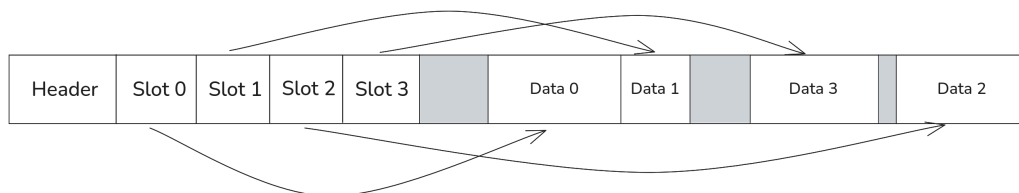


Figura 2: Layout de un `SlottedPage`. Nótese la fragmentación, la cual se repara a demanda con `compact()` cuando sea necesario.

Ambos layouts están parametrizados de forma genérica para admitir almacenar data adicional en el header de la página y en cada slot.

Los layouts se construyen a partir de un `PageGuard` como parámetro de constructor, el cual almacenan y usan internamente para sus operaciones.

Toda llamada a `BufferManager::fetch_page` en el proyecto se usa para construir alguno de los dos layouts; nunca se tocan los bytes crudos de una página fuera de los layouts.

3.1.4 Files

`SeqFile` y `AppendFile` son la capa de abstracción principal que permite interactuar con los archivos de registros, y definen su organización primaria. Cada uno se detalla más adelante en el informe.

3.1.5 Table

La clase `Table` es la abstracción final sobre el concepto de «tabla» en la base de datos. Contiene un `SeqFile` y la metadata de columnas e índices de la tabla que representa.

3.1.6 Catalog

La clase `Catalog` actúa como abstracción final de la base de datos como tal. Tiene las siguientes responsabilidades:

- Generar rutas para los archivos (tablas, índices, etc.).
- Crear y eliminar tablas e índices.
- Construir instancias de `Table` e índices a partir del sistema de archivos.

3.1.7 Query executor

El query executor (`QueryExecutor`, o executor para abreviar) recibe el AST directamente del parser y lo ejecuta en la base de datos bajo un `Catalog` ya instanciado. El executor implementado hace uso del patrón **volcano iterator**. Cada nodo del árbol de ejecución provee un método `next()` que retorna `std::optional<Row>` de forma *lazy*. Los métodos `open()` y `close()`, usualmente presentes en definiciones académicas del modelo, se implementan mediante el constructor y destructor de los nodos.

La transferencia de datos del executor al usuario del executor se hace mediante una interfaz `RowSink`, la cual define todos los métodos necesarios para transferir la información tupla por tupla.

Cabe destacar también que el DBMS solo soporta eliminación por igualdad en llaves primarias.

3.1.8 API Server

Es la capa final a la que se conecta un usuario en el frontend. Usa la librería `httplib`. Contiene los dos siguientes endpoints:

- `GET /`: Retorna un mensaje de estado.
- `POST /query`: Realiza una query a la base de datos. Debe contener las consultas a realizar como texto en el cuerpo de la petición.

Dado se tomó la decisión de enviar la data al cliente en formato JSON, no es posible actualmente enviar la data por *streaming*. El API Server usa una implementación de `RowSink` que simplemente acumula toda la información recibida en vectores en RAM, y esto es lo que se envía al cliente finalizadas la consultas. Sería posible que el backend devuelva data via *streaming* si, por ejemplo, se desarrollara un protocolo simple de transferencia por TCP explícitamente diseñado para enviar las tuplas resultantes por partes. Esta es una idea que tenemos en consideración para el desarrollo futuro del proyecto.

El esquema de respuesta de `POST /query` es el siguiente:

```
type Response = {
  reads: number,
  writes: number,
  time_ms: number,
```

```

    result: QueryResult[],
}

type QueryResult = {
    status: "ok" | "error",

    // si status == "ok"
    table: Table | null,
    plane: Plane | null,
    messages: string[],
    warnings: string[],

    // si status == "error"
    detail: string,
}

type Table = {
    columns: Column[],
    rows: Row[],
}

type Column = {
    name: string,
    type: "Int" | "Real" | "Bool" | "VarChar" | "Point2d",
}

type Row = Value[]

type Value = number | boolean | string | Point2D

type Plane = {
    rects: [number, Rect][],
    point: Point2D | null,
    radius: number | null,
}

type Rect = {
    min: PointD,
    max: Point2D
}

type Point2D = [number, number]

```

3.2 Técnicas adicionales de implementación

3.2.1 Cursores

Como la información que produce el DBMS debe ser transferida de forma incremental (por ejemplo, para permitir *streaming*), todo método de búsqueda de datos que pueda retornar más de un valor se implementa construyendo un «cursor»: una clase con un método `next()` que retorna el siguiente elemento encontrado (e.g. una tupla, un RID, etc.).

3.2.2 API de serialización/deserialización a bytes

El proyecto implementa una API similar al módulo `pack` de Python para serializar y deserializar objetos arbitrarios a memoria. Esta API se declara en `include/pack.hpp`. Consiste principalmente de tres clases:

```

template<typename T>
struct PackSize {
    std::size_t operator()(const T& val) const;
};

template<typename T>
struct Pack {
    void operator()(const T& val, u8*& dest) const;
};

template<typename T>
struct Unpack {
    void operator()(T& dest, const u8*& src) const;
};

```

Estas clases permiten que un objeto se pueda serializar («pack») y deserializar («unpack»). El mecanismo se puede implementar para un tipo cualquiera de C++ por medio de especialización. El archivo `include/pack` incluye algunas implementaciones genéricas para tipos trivialmente copiables, strings, vectores y variants.

Como mejora por encima del módulo `struct` de Python, esta API de pack/unpack permite la serialización a un **tamaño variable** de bytes (por ejemplo, para strings y vectores).

3.2.3 Uso de features de C++ moderno

Una de las adiciones más importantes al proyecto ha sido el uso de `gcc c++23`, esto nos ha dado acceso a dos funcionalidades fundamentales para el manejo de errores empleado a lo largo del código, `std::expected` y los *bloques como expresiones*. `std::expected` ha sido nuestra forma primaria de propagar y manejar errores a lo largo del código; sus semánticas de error como valor han hecho mucho más explícito el manejo de errores. Sin embargo, un problema que encontramos inicialmente con `std::expected` es que era especialmente poco ergonómico para *propagar* errores; `c++` sigue dependiendo de excepciones para propagar errores de forma cómoda, pero una solución que se le encontró a esto fue una extensión del compilador `gcc` que permite usar bloques como expresiones, esta funcionalidad se usa a lo largo del proyecto mediante las macros **TRY** que se pueden encontrar en `/include/util.hpp`; esta adición mejoró enormemente la ergonomía de `std::expected` y lo hizo finalmente viable como forma primaria de manejo de errores. A continuación se muestra un ejemplo de las macros **TRY** y su uso en código:

```

#define TRY(expr) \
    ({ \
        auto&& res = (expr); \
        if (!res) \
            return std::unexpected(res.error()); \
        std::move(*res); \
    })

// TRYV es un equivalente a TRY, pero no retorna el valor contenido en res; se usa
// principalmente para propagar std::expected<void, Err>
#define TRYV(expr) \
    ({ \
        auto&& res = (expr); \
        if (!res) \
            return std::unexpected(res.error()); \
    })

// nótese que, para poder hacer uso de las macros TRY, el tipo de retorno de la función
// debe ser un std::expected con el mismo tipo de error que el objeto a propagar
std::expected<DeleteStatement, CompileError> Parser::delete_statement() {

```



```

TRYV(expect_val<Keyword::Delete>());
TRYV(expect_val<Keyword::From>());

// aquí se usa TRY para volver el std::expected<Identifier, CompileError> retornado
// por expect_var un Identifier; en caso expect_var retorne CompileError, la macro se
// encargará de que la función retorne en este punto y el error se propagará
auto table_tok = TRY(expect_var<Identifier>());

// la macro TRY es equivalente a repetir este patrón:
// auto res = expect_var<Identifier>();
// if (!res.has_value()) return std::unexpected{res.error()};
// auto table_tok = *res;

DeleteStatement stmt{};
stmt.table_name = table_tok.value;

TRYV(expect_val<Keyword::Where>());

auto iden_tok = TRY(expect_var<Identifier>());
stmt.col_name = iden_tok.value;

TRYV(expect_val<Symbol::Eq>());
stmt.value = TRY(expr_lit());
TRYV(expect_val<Symbol::SemiColon>());

return stmt;
}

```

4 Técnicas implementadas

4.1 Parsing

Se implementó un parser de SQL usando la técnica `recursive descent`. A continuación se presenta la gramática usada en el proyecto en notación EBNF.

```
source_file := { statement } ;
```

```
statement := create_statement |
            select_statement |
            insert_statement |
            delete_statement |
            drop_statement |
            show_statement ;
```

```
create_statement :=
  "create" "table" [ "if" "not" "exists" ] identifier "("
  column_decl { "," column_decl } ")" [ "from" "file" STR_LIT ] ";"
;
```

```
column_decl := identifier datatype [ ( "index" identifier ) | ( "primary" "key" ) ]
```

```
select_statement :=
  "select" "*" "from" identifier [ "where" where_declaration ] ";"
;
```

```
insert_statement :=
  "insert" "into" identifier "values"
  "(" insert_value ")"
```

```

{ "," "(" insert_value ")" } ";"
;

delete_statement :=
  "delete" "from" identifier "where" identifier "=" expr_lit ";"
;

drop_statement :=
  "drop" "table" [ "if" "exists" ] identifier ";"
;

show_statement :=
  "show" "index" identifier "." identifier ";"
;

where_declaration :=
  identifier (
    ( "=" expr_lit )
    ( "between" expr_lit "and" expr_lit )
    ( "in" "(" "point" "(" NUM_LIT "," NUM_LIT ")" "," ( ( "radius" NUM_LIT ) | ( "k"
NUM_LIT ) ) ")" )
  )
;

expr_lit :=
  STR_LIT |
  NUM_LIT |
  "true" |
  "false" |
  "point" "(" NUM_LIT "," NUM_LIT ")"
;

```

En general, el procesamiento de sintaxis se dividió en dos partes:

4.1.1 Escaneo

Para el escaneo se utilizó un *autómata finito determinístico*, el cual se presenta en la siguiente tabla de transiciones:

	"	'	(	)	*	+	,	-	.	/	;	<	=	>	\	¹	²	³	⁴	⁵
q0	q20	q22	q10	q11	q1	q13	q2	q15	q12	q3	q14	q7	q4	q5			q19		q16	
q5													q6							
q7													q8	q9						
q15								q25											q16	
q3					q27															
q27					q28											q27				
q28										q26						q27				
q25																q25				q26

¹cualquier caracter

²cualquier letra o el caracter “_”

³cualquier letra o número o caracter “_”

⁴cualquier número

⁵salto de línea

q16									q17									q16	
q17																		q18	
q18																		q18	
q19																q19			
q20	q21																q20		
q22		q24												q23	q22				
q23															q22				

La detección de *keywords* y *data types* se realiza luego del escaneo mediante tablas generadas a partir de los enums **Keyword** y **DataType** usando la librería **magic_enum**.

4.1.2 Parseo

El parseo se realizó con la técnica de *descenso recursivo*. Dado que el parseo recibe a demanda tokens generados por el escáner, se utilizó una abstracción **Peekable<T>** que utiliza el escáner como un iterador y permite inspeccionar el objeto actual sin consumirlo (parecido al **operator*** de los iteradores de C++). El proceso de validar los tokens retornados por el escáner se abstraio con funciones auxiliares de **expect** y **accept**. Otra herramienta muy presente en el parser (y otros componentes del proyecto) son las macros **TRY**, estas utilizan una extensión de GCC que permite utilizar scopes como expresiones. Esta es una de las adiciones más importantes para el manejo de errores a lo largo del proyecto y se usa a lo largo de todo el código, principalmente porque la forma primaria de representar y propagar errores usada en el proyecto es con la clase **std::expected** de C++ 23.

4.2 Archivos de registros

4.2.1 SeqFile

El proyecto implementa una clase **SeqFile** como estructura principal para almacenar físicamente las tuplas de una tabla. Utiliza la técnica vista en clase de usar un **espacio auxiliar** no ordenado. Cuando este espacio se llena, se trae a memoria RAM, se ordena, y el archivo entero se reconstruye con el algoritmo de combinar dos listas ordenadas.

El espacio auxiliar se define como una página adicional, y su capacidad es todo lo que pueda contener hasta una inserción que no quepa en el espacio restante (ya que los registros son de tamaño variable). La razón por la que se optó por un tamaño constante de página auxiliar fue porque:

1. Usar una página auxiliar de tamaño k dependiente de la cantidad de registros n , por más que fuera algo lentamente creciente como $k \in O(\log(n))$, implicaría que k no tendría cota superior.
2. Por lo tanto, un ordenamiento en RAM del espacio auxiliar sería una mala decisión de diseño, ya que no existiría una cota sobre la cantidad de datos que se traerían a RAM. Una opción viable en ese caso sería usar **external sort** para cuando k sea lo suficientemente grande, lo cual quedó fuera del alcance del proyecto.

Sin embargo, la decisión de usar un espacio auxiliar de tamaño constante tiene una fuerte desventaja: las inserciones en el sequential file no son $O(1)$ amortizado, sino $O(n)$, ya que el tamaño constante del espacio auxiliar solamente acelera la inserción por un factor constante.

Se selecciona este tipo de archivo automáticamente al crear una tabla con llave primaria.

4.2.2 AppendFile

Se implementó también un tipo de archivo denominado **AppendFile**, el cual solo permite inserciones, mas no eliminaciones. Su implementación es trivial: siempre inserta al final del archivo, en la primera página con espacio libre. Hace uso de **SlottedPage**.

Este tipo de archivo fue creado con el propósito específico de realizar benchmarks de selección de datos con índices. El sequential file es tan lento en su inserción, que resulta imposible insertar cantidades de datos suficientes como para realizar benchmarks útiles de consultas **SELECT**. El **AppendFile** fue creado como solución provisional para al menos poder realizar estos benchmarks sobre los índices, donde no es relevante la organización primaria del archivo.

Se selecciona este tipo de archivo automáticamente al crear una tabla sin llave primaria. No pretende tener un uso útil en el DBMS en la práctica mas allá del benchmarking.

4.3 Índices

Al igual que los archivos de registros, los índices acceden a memoria secundaria exclusivamente mediante el buffer manager y las abstracciones de los layouts. Todos los algoritmos descritos realizan implícitamente sus lecturas/escrituras sobre instancias de **SlottedPage** y **RecordPage**, las cuales, como se detalló anteriormente, gestionan automáticamente la escritura eventual a disco. Asimismo, cuando se habla de un «bucket», (en el caso del índice hash) o un «nodo» (en el caso de los árboles), implícitamente estos se refieren a páginas en disco que albergan estas estructuras, y son identificadas por un **número de página** (`pnum_t`).

4.3.1 Extendible Hashing

Para el proyecto se implementó un índice *hash* para tipos booleanos, enteros y strings. Por debajo utiliza el esquema de *extendible hashing* con profundidad máxima de 8, el máximo n tal que un array directorio de 2^n entre en la página cabecera del archivo (asumiendo 4 KB de página). En el header del índice se almacenan las direcciones de los buckets y la profundidad actual, mientras que cada bucket almacena o bien su profundidad local (dependiendo de cuántos splits se han realizado sobre dicho bucket) o la dirección al siguiente bucket encadenado (en caso se haya superado la profundidad máxima y se tenga que usar una lista enlazada).

El procedimiento de inserción, entonces, es como prosigue:

1. Se comienza hasheando la llave y obteniendo el main bucket correspondiente al hash, módulo 2^D (sea D la profundidad global actual). Se comienza considerando este bucket.
2. Si la llave-valor a insertar entran en el bucket actual, insertar y culminar el procedimiento.
3. Si el bucket actual está encadenado (contiene el `pnum_t` del siguiente bucket):
 1. Si el siguiente bucket es nulo (`PAGE_NIL`), alojar un nuevo bucket y encadenarlo al actual.
 2. Considerar ahora el siguiente bucket encadenado. Ir al paso 2.
4. El bucket actual es un bucket principal sin encadenar. Intentar incrementar su profundidad local.
5. Si la nueva profundidad local de este bucket supera a la profundidad global máxima, no hacer split. En su lugar, encadenar el bucket con `PAGE_NIL` y, todavía considerando a este mismo bucket, ir al paso 3.
6. Si la nueva profundidad local supera a la profundidad global, actualizar la profundidad global y duplicar el directorio.
7. Alojar 2 nuevas páginas para buckets y usarlos para splitear el bucket actual.
8. Liberar la página del bucket original.
9. Considerar ahora, de los 2 buckets del split, aquel correspondiente al bit adicional ahora considerado con la nueva profundidad local. Ir al paso 2

El procedimiento de búsqueda es comparativamente mucho más sencillo: se hashea la llave a buscar, módulo 2^D , y se crea un cursor que escanea el bucket correspondiente a aquel valor.

El caso de eliminación es similar: se hace el procedimiento de hash con módulo, se busca el bucket principal correspondiente, y se busca al primer elemento que coincida en llave y valor. De encontrarse, se elimina dicho slot haciendo *move-the-last* con el último slot del bucket. La implementación no aborda el desalojar buckets vacíos o ideas similares.

4.3.2 B+ Tree

El *B+ tree* (no agrupado, por ser un índice) implementado en el proyecto sigue las mismas abstracciones que el índice *hash*. El índice en disco posee un header que guarda el número de página correspondiente a la raíz del árbol. Cada nodo posee un header indicando su último hijo (usado por nodos internos, representa el único hijo derecho del nodo) y las página previa y siguiente (para enlazar las hojas). Cada slot, aparte del rid asociado a su llave, almacena su hijo izquierdo. En lugar de usar un indicador de `is_leaf`, los nodos hojas se distinguen cuando `last_child == PAGE_NIL`.

Este índice B+ Tree soporta llaves de tamaño variable por medio del `SlottedPage`. Por ende, no posee un parámetro M particular, sino que realiza rebalanceos en función del **porcentaje espacio ocupado** (en bytes) del nodo, excluyendo el header del mismo. En particular, se utiliza `page.total_space() / 2` como threshold de espacio en los rebalanceos.

Al igual que el *hash*, se implementó un *RangeCursor* para la búsqueda dado que se admiten valores repetidos en el índice.

Un detalle a señalar del algoritmo de eliminación es que, dado que el proceso de búsqueda encuentra el primer elemento con la llave a buscar y luego itera linealmente hasta encontrar el RID correcto, el elemento a eliminar podría estar en un nodo distinto al primer nodo encontrado. Esto se resolvió sobrescribiendo el RID del slot a eliminar con el RID del primer slot encontrado entre las hojas. Esto facilita la implementación al permitir realizar el algoritmo de eliminación habitual como si estuviéramos eliminando el primer elemento con la llave indicada. La Figura 3 muestra un ejemplo de este procedimiento.

4.3.3 R-Tree

El proyecto presenta una implementación propia de R-Tree en disco. Aunque está parametrizada por un número arbitrario de dimensiones N , el DBMS desarrollado usa únicamente el R-Tree de 2 dimensiones. El proyecto también implementa un tipo de dato adicional, **Point2D**, diseñado específicamente con el R-Tree en mente.

Para la inserción, se utiliza el algoritmo de partición cuadrática, el cual selecciona los dos nodos que generan menor diferencia de volumen al combinarse como semillas, y después se asignan las semillas restantes según el nodo con el cual generan la menor cantidad de alargamiento.

Para la eliminación, similar a como ocurre con un árbol B, se controla el caso en el que un nodo tenga menos de $\lfloor \frac{MAX}{2} \rfloor$ llaves, en el cual se reinsertan los hijos a la misma profundidad de la que fueron eliminados. La reinserción se propaga recursivamente desde las hojas.

Para la búsqueda KNN, se utiliza un algoritmo basado en heaps el cual aprovecha el filtro de los rectángulos para explorar únicamente los nodos necesarios. Se hace uso de la distancia euclidiana.

5 Resultados y análisis

Las pruebas presentadas a continuación se realizaron con el siguiente dataset de Kaggle: <https://www.kaggle.com/c/nyc-taxi-trip-duration/data>. Dado que nuestro DBMS establece que un dato `point2d` en CSV debe tener el formato `<x>;<y>`, el dataset se modificó combinando las columnas `pickup_longitude`, `pickup_latitude` y `dropoff_longitude`, `dropoff_latitude` respectivamente para generar las columnas `pickup_point` y `dropoff_point`.

Tras la transformación, el dataset tiene las siguientes columnas:

- `id`: varchar

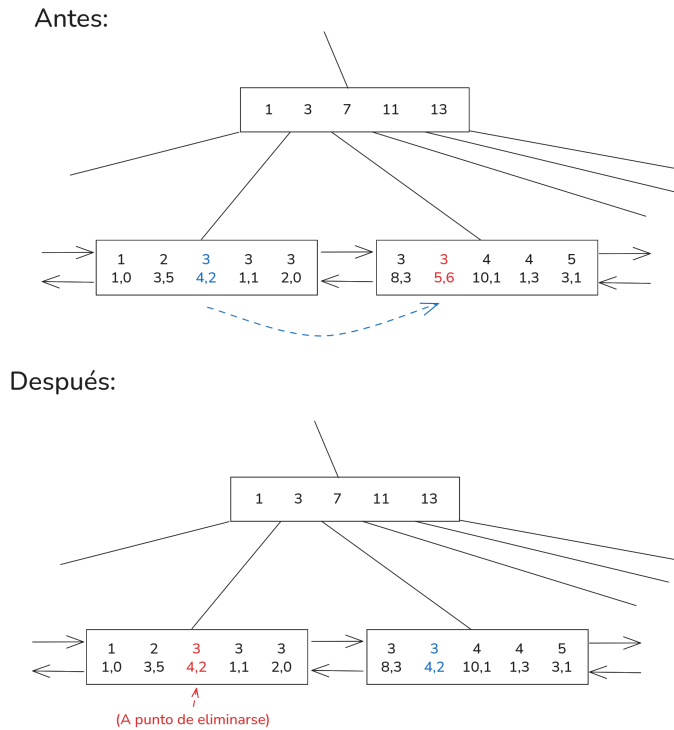


Figura 3: Ejemplo de una eliminación de llave 3 y RID (5,6) en un índice B+ Tree. Primero, se baja hacia el primer slot hoja de llave 3. Luego, se recorren las hojas secuencialmente hasta encontrar el RID (5, 6). Finalmente, se sobrescribe el RID a eliminar, (5,6) con el primer RID encontrado (4,2). Luego de esto se procederá a eliminar el nodo que queda en rojo en la segunda imagen.

- `vendor_id`: int
- `pickup_datetime`: varchar
- `passenger_count`: int
- `pickup_point`: point2d
- `dropoff_point`: point2d
- `store_and_fwd_flag`: varchar

Todos los archivos CSV usados en el proyecto se encuentran en `data/sample`. Además de conservar el dataset original, de ~600k tuplas, en `uber_full.csv`, se crearon versiones truncadas de los datasets para distintos tamaños: `uber_100.csv`, `uber_1k.csv`, `uber_10k.csv` y `uber_100k.csv`.

5.1 I/O y tiempos de inserción

Esta sección evalúa la cantidad de I/O y el tiempo de inserción del sequential file con espacio auxiliar constante.

5.2 I/O y tiempos de búsqueda

Esta sección evalúa la cantidad de I/O y el tiempo de búsqueda con cada índice implementado (puntual y por rango). Estos benchmarks hacen uso del `AppendFile`, ya que la estructura primaria del archivo tiene impacto despreciable cuando lo que se evalúa son índices.

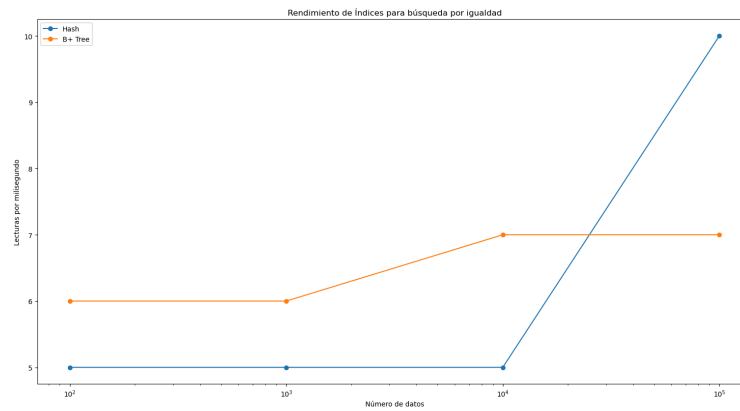
Las consultas realizadas para búsquedas por igualdad fueron las siguientes:

```
select * from uber_* where passenger_count = 6; -- Sin índice
select * from uber_* where pickup_datetime = '2016-06-30 23:37:07'; -- Hash
select * from uber_* where id = 'id2126329'; -- B+ Tree
```

Los datos se presentan en formato <lecturas> / <tiempo>.

Tabla 1: Resultados experimentales de búsquedas por igualdad usando índices hash y B+ Tree.

Índice	100 datos	1k datos	10k datos	100k datos
Ninguno	7 / 4 ms	44 / 8 ms	419 / 69 ms	4169 / 1096 ms
Hash	5 / 1 ms	5 / 1 ms	5 / 1 ms	10 / 1 ms
B+ Tree	6 / 1 ms	6 / 1 ms	7 / 1 ms	7 / 1 ms

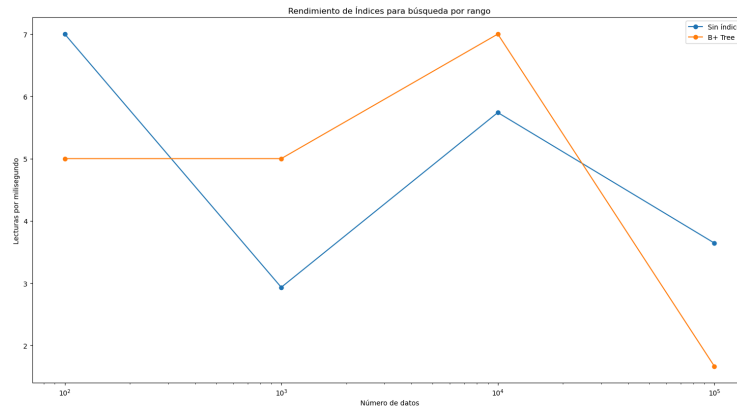


Las consultas para búsquedas por rango son las siguientes:

```
select * from uber_* where passenger_count between 6 and 8; -- Sin índice
select * from uber_* where id between 'id0928000' and 'id0928400'; -- B+ Tree
```

Tabla 2: Resultados experimentales de búsquedas por rango usando índices hash y B+ Tree.

Índice	100 datos	1k datos	10k datos	100k datos
Ninguno	7 / 1 ms	44 / 15 ms	419 / 73 ms	4169 / 1144 ms
B+ Tree	5 / 1 ms	5 / 1 ms	7 / 1 ms	15 / 9 ms



Las consultas para el índice R-Tree son las siguientes:

```
select * from uber_* where pickup_point in (point(-50, 50), radius 10);
```

5.2.1 Análisis

Notamos que para datos estándar, tanto el índice Hash como el índice B+ tienen un impacto significativo en la velocidad de consultas, en especial a medida que aumenta el volumen de datos. Para datos espaciales, nuevamente el índice R-Tree optimiza notoriamente los tiempos de ejecución, debido a la búsqueda reducida del índice la cual permite reducir la cantidad de consultas.

Algo importante de mencionar es que, en la búsqueda por rango, el índice btree presentó resultados irregulares frente a una consulta por rango sin índice. Esto muy probablemente se debe a que se están usando diferentes columnas.

5.3 Conflictos por concurrencia

Dado que la librería `cpp-http-lib` usada para la conexión con el frontend instancia threads automáticamente para manejar las solicitudes, el sistema ya es completamente capaz de manejar consultas en simultáneo (aunque sin ningún mecanismo de detección y manejo de conflictos). A continuación se ilustra un caso de prueba que demuestra la lectura y escritura concurrente a archivos mediante ejecución simultánea de 2 consultas (para referencia de las consultas ejecutadas, ver anexo):


```

Server listening on 0.0.0.0:8080
<139774939625152> begin
<139774931232448> begin
<139774939625152> read page 1 from .data/users.p.index.bin
<139774931232448> read page 1 from .data/users.p.index.bin
<139774939625152> write page 1 to .data/users.p.index.bin
<139774931232448> write page 1 to .data/users.p.index.bin
<139774939625152> read page 1 from .data/users.table.bin
<139774931232448> read page 1 from .data/users.table.bin
<139774939625152> write page 1 to .data/users.table.bin
<139774931232448> write page 1 to .data/users.table.bin
<139774939625152> read page 1 from .data/users.table.bin
<139774931232448> read page 1 from .data/users.table.bin
<139774939625152> write page 1 to .data/users.table.bin
<139774931232448> write page 1 to .data/users.table.bin
<139774939625152> read page 1 from .data/users.p.index.bin
<139774931232448> read page 1 from .data/users.p.index.bin
<139774939625152> write page 1 to .data/users.p.index.bin
<139774931232448> write page 1 to .data/users.p.index.bin
<139774939625152> write page 1 to .data/users.table.bin
<139774931232448> write page 1 to .data/users.table.bin
<139774939625152> write page 1 to .data/users.p.index.bin
<139774931232448> write page 1 to .data/users.p.index.bin
<139774939625152> write page 1 to .data/users.table.bin
<139774931232448> write page 1 to .data/users.table.bin
<139774939625152> write page 1 to .data/users.p.index.bin
<139774939625152> end
<139774931232448> write page 1 to .data/users.p.index.bin
<139774931232448> end
<139774922839744> begin
<139774922839744> read page 1 from .data/users.table.bin
<139774922839744> end

```

Figura 6: Se observan principalmente conflictos **write-write** entre la primera y segunda consulta

CONSOLA SQL KOAKUMA ENDPOINT http://localhost:8080

```
1 select * from users;
```

EJECUTAR

LECTURAS: 3 ESCRITURAS: 0 TIEMPO: 1000 MS

Id	P
4	(12.-2, 30.5)
5	(12.-2, 30.5)
6	(12.-2, 30.5)

Figura 7: Se verifica que la última escritura eliminó la escritura anterior

6 Evidencias de la interfaz

El proyecto completo se puede ejecutar con el `docker-compose.yml` presente en el repositorio del DBMS. A continuación se muestra el funcionamiento general del frontend (las consultas ejecutadas se encuentran en el anexo):



Figura 8: Consola del frontend para hacer consultas

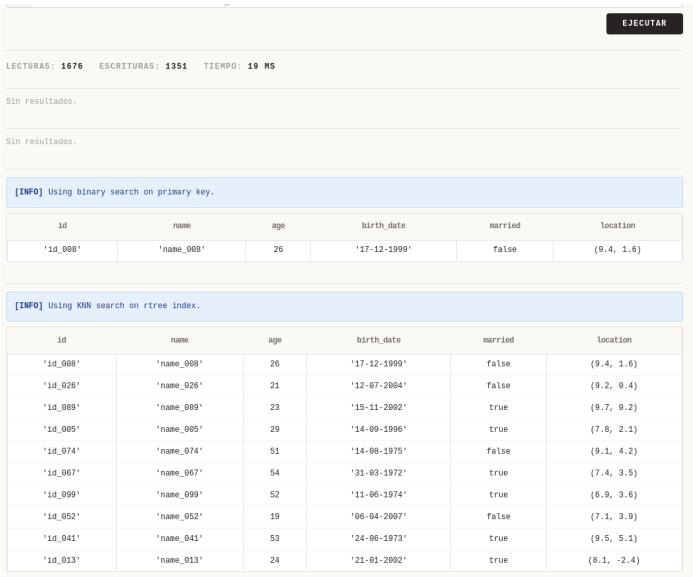


Figura 9: Muestra de consultas de la Figura 8 ejecutándose.

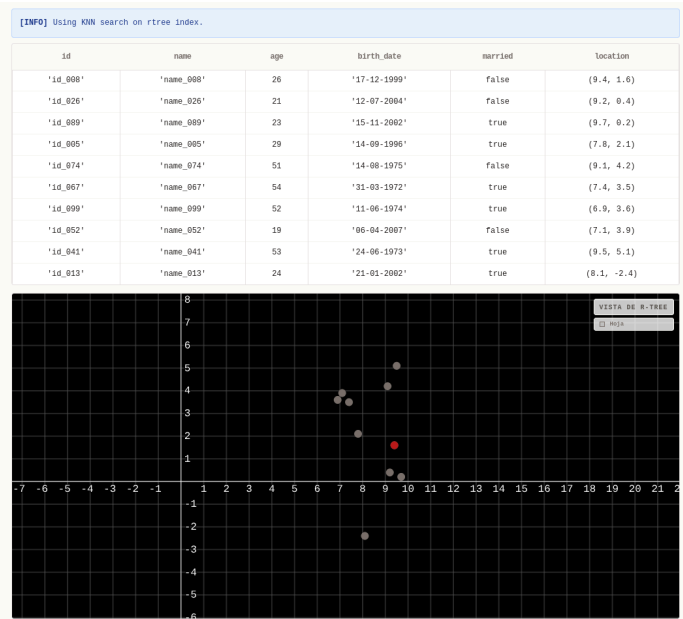


Figura 10: Visualización del índice R-tree en consulta KNN

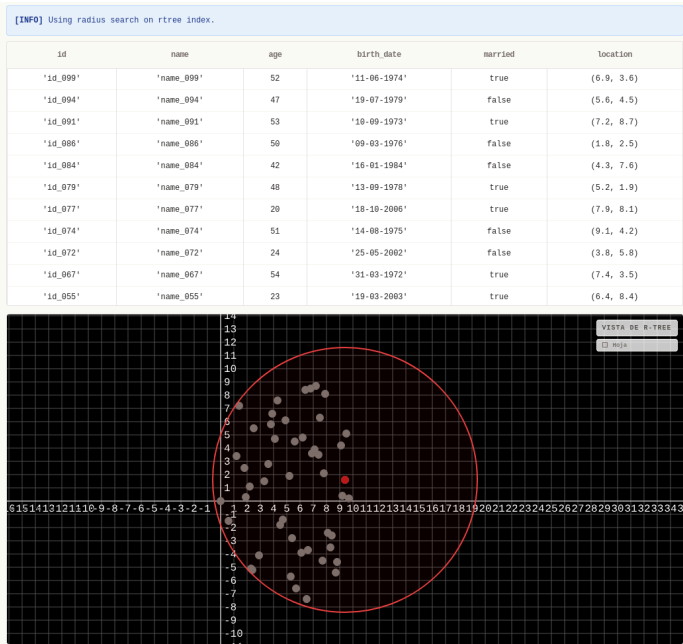


Figura 11: Visualización del índice R-tree en búsqueda por radio

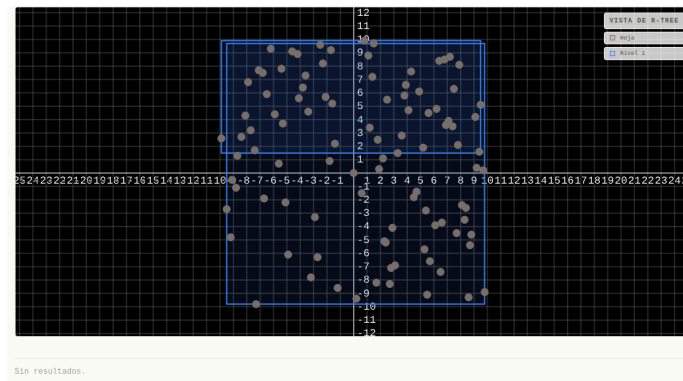


Figura 12: Visualización general del índice R-tree con la consulta `SHOW INDEX`

7 Anexo

7.1 Consulta de ejemplo

-- Consultas de ejemplo

```
drop table if exists users;
```

-- Tipos: int, real, bool, varchar, point2d

-- Índices: hash, btree, rtree

```
create table users (
  id          varchar primary key,
  name        varchar index hash,
  age         int index btree,
  birth_date  varchar,
  married     bool index hash,
  location    point2d index rtree
) from file 'data/sample/users.csv';
```

-- Usuarios adicionales

```
insert into users values
  ('user_00', 'juan', 42, '01/01/2000', true, point(2, 3)),
  ('user_01', 'carlos', 19, '01/01/2004', false, point(0, -2.5));
```

```
select * from users where id = 'id_087';
```

```
select * from users where married = true;
```

```
select * from users where location in (point(9.4, 1.6), k 10);
```

```
select * from users where location in (point(9.4, 1.6), radius 12);
```

```
show index users.location;
```

7.2 Consultas para crear tablas con el dataset de Kaggle

```
drop table if exists uber_100;
```

```
drop table if exists uber_1k;
```

```
drop table if exists uber_10k;
```

```
drop table if exists uber_100k;
```

```
create table if not exists uber_100 (
```

```

    id varchar index btree,
    vendor_id int,
    pickup_datetime varchar index hash,
    passenger_count int,
    pickup_point point2d index rtree,
    dropoff_point point2d index rtree,
    store_and_fwd_flag varchar
) from file 'data/sample/uber_100.csv';

create table if not exists uber_1k (
    id varchar index btree,
    vendor_id int,
    pickup_datetime varchar index hash,
    passenger_count int,
    pickup_point point2d index rtree,
    dropoff_point point2d index rtree,
    store_and_fwd_flag varchar
) from file 'data/sample/uber_1k.csv';

create table if not exists uber_10k (
    id varchar index btree,
    vendor_id int,
    pickup_datetime varchar index hash,
    passenger_count int,
    pickup_point point2d index rtree,
    dropoff_point point2d index rtree,
    store_and_fwd_flag varchar
) from file 'data/sample/uber_10k.csv';

create table if not exists uber_100k (
    id varchar index btree,
    vendor_id int,
    pickup_datetime varchar index hash,
    passenger_count int,
    pickup_point point2d index rtree,
    dropoff_point point2d index rtree,
    store_and_fwd_flag varchar
) from file 'data/sample/uber_100k.csv';

```

7.3 Consultas ejecutadas en la demostración de acceso concurrente

7.3.1 Primera Consulta

```

create table if not exists users (
    id int primary key,
    p point2d index rtree
);

insert into users values
    (1, point(12.20, 30.5)),
    (2, point(12.20, 30.5)),
    (3, point(12.20, 30.5));

```

7.3.2 Segunda Consulta

```

create table if not exists users (
    id int primary key,
    p point2d index rtree
);

```

```
insert into users values
  (4, point(12.20, 30.5)),
  (5, point(12.20, 30.5)),
  (6, point(12.20, 30.5));
```

7.4 Consultas ejecutadas en la demostración de la interfaz

```
drop table if exists users;
```

```
create table users (
  id          varchar,
  name        varchar index hash,
  age         int index btree,
  birth_date  varchar,
  married     bool index hash,
  location    point2d index rtree
) from file 'data/sample/users.csv';
```

```
select * from users where id = 'id_008';
select * from users where location in (point(9.4, 1.6), k 10);
select * from users where location in (point(9.4, 1.6), radius 10);
show index users.location;

delete from users where id = 'id_008';
```